

52. Nommer un geste qu'on refait

Une fonction avec un argument, et trois appels.

1. Ce qu'il faut comprendre

Les fonctions prennent des arguments, et c'est là tout l'intérêt. Sans argument, il faudrait trois fonctions, ou une variable globale à poser avant chaque appel.

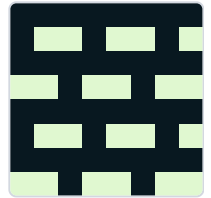
`void` dit qu'elle ne rend rien. Le compilateur le vérifie : une fonction qui annonce rendre un `uint8_t` sans jamais faire `return` est **refusée, avec sa ligne**.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1366 octets** de cartouche.

1. UN DESSIN — BRIQUE

```
Tuile BRIQUE = {  
    "#####", "#..#..#.", "#####", "..#..#..",  
    "#####", "#..#..#.", "#####", "..#..#..",  
};
```



à l'écran

2. UNE FONCTION — RANGEE()

```
void rangee(uint8_t ligne) {  
    for (uint8_t x = 0; x < 20; x++) poser(x, ligne, BRIQUE);  
}
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    rangee(4);  
    rangee(9);  
    rangee(14);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois rangées de briques, écrites par un seul bout de code.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première rangée est posée
- ✓ la deuxième aussi
- ✓ et la troisième
- ✓ rien entre les rangées

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

 **Game Boy — 4 nuances**Les quatre nuances de la console d'origine, et rien de plus.

 Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

- zoom

22×

+ zoom

 Plein écran

 Peindre ✓

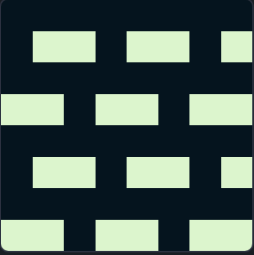
 Sélectionner et déplacer

 Copier le dessin

 Coller ici

 Dupliquer

 Effacer la tuile





taille réelle — 8 × 8 pixels

Ensuite : **53. Une fonction qui rend une valeur** — niveau 7 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++